

Rapport Projet C++ - Dog Food Dispenser

Le but de ce projet est de réaliser un distributeur de croquettes pour chien, capable de gérer plusieurs chiens à la fois via des badges RFID. La distribution des croquettes ne doit pas être à heure fixe mais doit respecter un délai minimal de 12 heures entre deux distributions et ce pour chaque chien.

Le distributeur doit également être accessible à distance via une interface web afin de pouvoir suivre en temps réel les données de distribution pour chaque chien ainsi que vérifier le stock de croquettes.

L'intérêt est de créer un distributeur autonome permettant aux utilisateurs de partir de chez eux, en week-end ou en vacances, sans être coincés avec la distribution de la nourriture de leurs chiens tout en étant en mesure de suivre les données afin de s'assurer que leurs animaux ne manquent de rien.

Scénarios et diagramme de séquence :

Un chien présente sa tête à travers l'ouverture du distributeur, ceci déclenche un front au niveau d'un capteur infra-rouge, le distributeur déclenche donc une interruption qui vérifie la quantité de croquettes restantes via un capteur ultrason, c'est-à-dire estime une quantité de croquette en fonction de la distance mesurée. Ensuite, si le niveau de croquettes est suffisant, une lecture RFID est déclenchée et le distributeur vérifie si le badge correspond à l'un des chiens connus. Si le badge est connu, le distributeur va alors vérifier dans les données du chien si ce dernier a été nourri il y a moins de douze heures. Si ce n'est pas le cas, le distributeur déclenche alors une distribution de croquettes via l'ouverture d'un servomoteur.

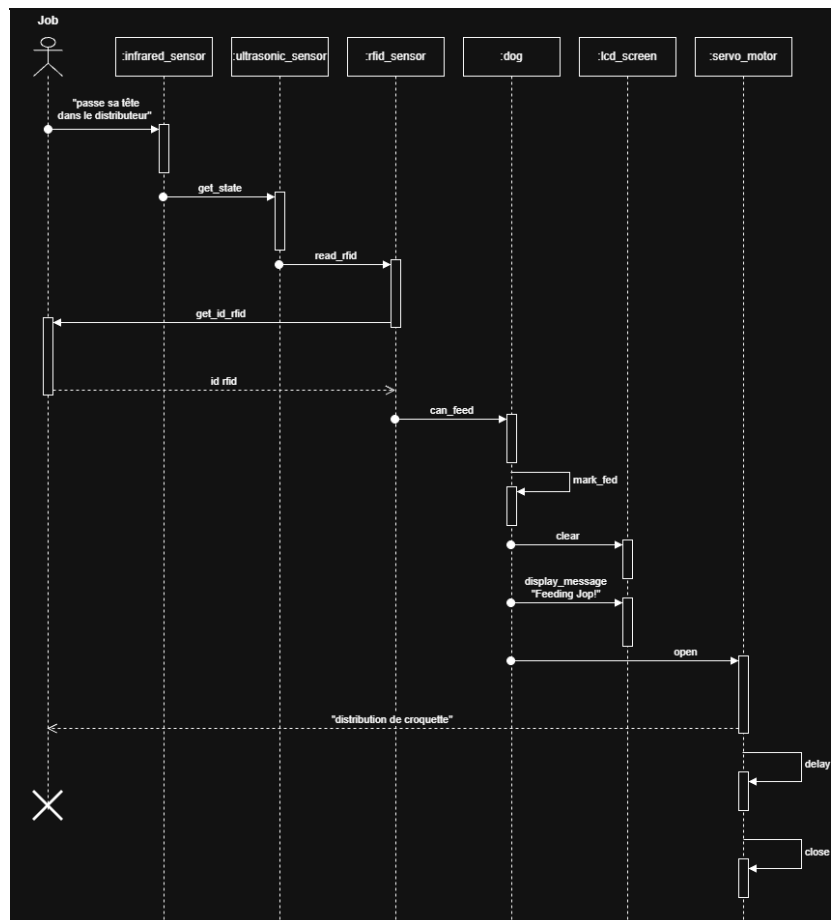


Figure 1 - Diagramme de séquence

Namespaces et diagrammes des classes

La librairie est divisée en namespaces afin de rendre plus compréhensible le code.

Dans chacun de ces namespace sont ensuite déclarées les différentes classes, variables et constantes nécessaires au bon fonctionnement du distributeur.

Figure 2 - Architecture des namespaces

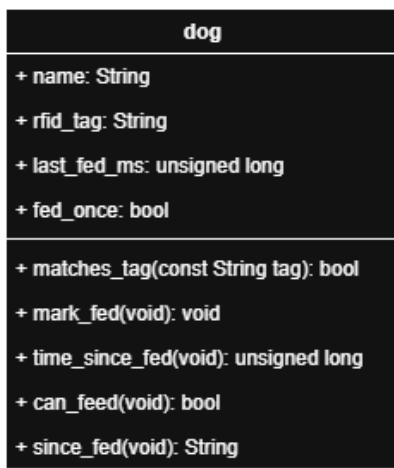
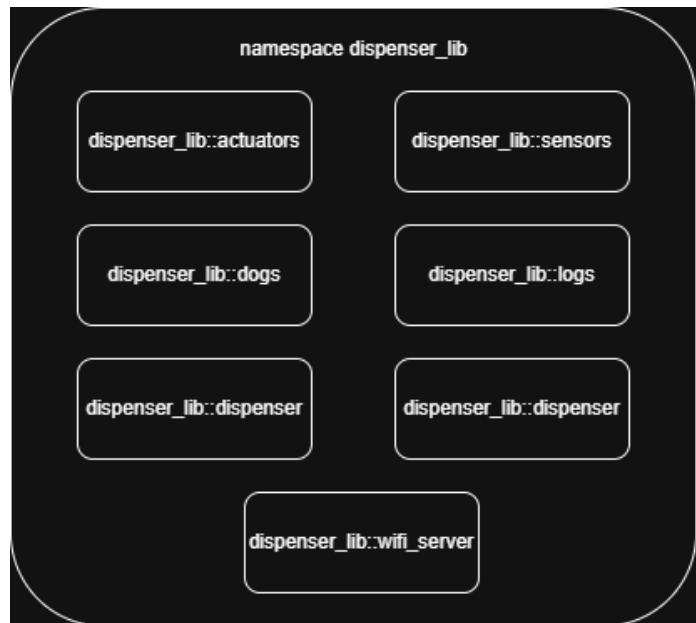


Figure 3 - Classe dogs

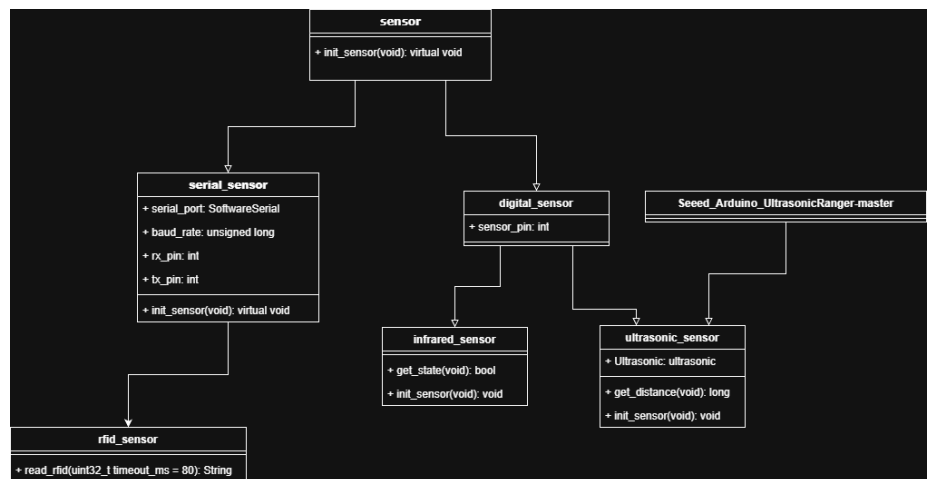


Figure 4 - Diagramme de classe du namespace dispenser_lib::sensors

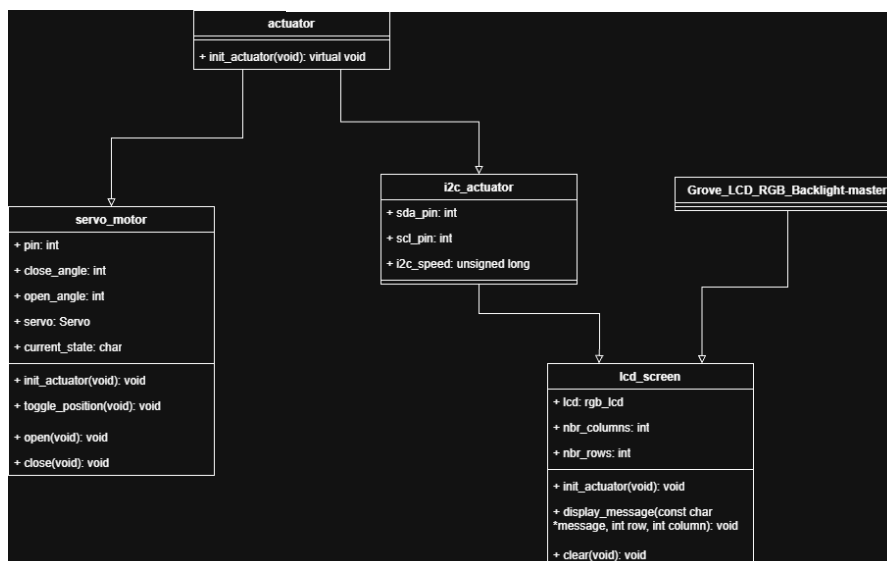


Figure 5 - Diagramme de classe du namespace dispenser_lib::actuators

Déroulement du projet

Pour ce projet nous avons commencé par réfléchir à toutes les fonctionnalités dont nous avons besoin pour ce distributeur. Ainsi nous avons listé les fonctionnalités et réfléchi aux capteurs et actionneurs dont nous aurions besoin afin de pouvoir les respecter. Une fois cela fait, nous avons commencé par coder des exemples pour chaque capteur et actionneur afin de comprendre comment ces derniers fonctionnaient et pouvoir les mettre en place plus facilement par la suite. Une fois un exemple codé pour chaque capteur et actionneur, nous avons commencé à réfléchir à comment nous voulions structurer notre code, c'est-à-dire les différents namespace, la répartition des différents fichiers et des différentes classes. Une fois cette répartition faite, nous sommes passé à la réalisation du premier code. Nous avons donc créé les premiers fichiers et codé les premières classes puis les avons testées ensemble pour la première fois. Ensuite, nous avons continué à coder et tester jusqu'à obtenir un premier jet de distributeur complet, à ce moment nous n'avions que le distributeur tel quel, sans page web ni système de logs. Après avoir peaufiné ce premier code, nous avons décidé d'ajouter des fonctionnalités afin de correspondre aux contraintes du sujet (surcharge d'opérateurs, lib STL ,etc..). Ainsi des nouvelles idées ont émergé afin de pouvoir correspondre aux contraintes imposées, le système de log a été réalisé afin de pouvoir ajouter une surcharge d'opérateur et utiliser la lib STL. La page web, à elle été ajoutée car nous avions du temps supplémentaire, nous avons donc réalisé la partie réseau et avons utilisé une IA (Grok) afin de réaliser l'esthétique de la page WEB car nous n'avions pas les connaissances en HTML, CSS et Javascript. Une fois le distributeur terminé, nous avons réalisé la documentation au format Doxygen à l'aide de l'IA intégrée à VS Code : Github Copilot. Nous avons alors ajouté le fait de pouvoir générer une page web de documentation, cela rendant la lecture de la documentation bien plus simple. Une fois cela fait, le projet était terminé.

Comment faire marcher le système

Pour faire marcher le système, il faut suivre les indications du fichier README.md afin de pouvoir compiler le projet et le flasher sur le microcontrôleur ESP8266. Il faut également brancher les différents capteurs et actionneurs sur la shield. Ensuite, pour tester le programme, il faut suivre les étapes indiquées par le diagramme de séquence.

Si vous voulez utiliser la page web, il faudra modifier le nom du WIFI et le mot de passe dans le code afin que la carte tente de se connecter au bon réseau

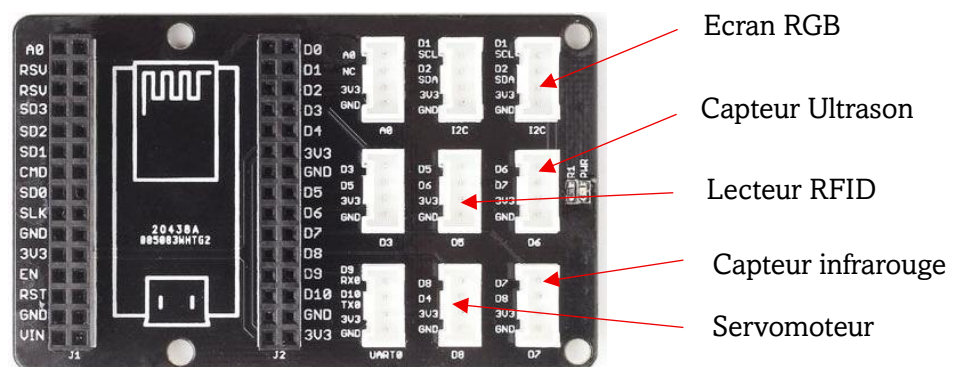


Figure 6 - Schéma des branchements des capteurs et actionneurs